Modification to STAtten for Overlapping Temporal Blocks

Brandon Esebag
California State Polytechnic University, Pomona
3801 W Temple Ave, Pomona, CA 91768
besebag@ieee.org

Abstract

Spatial-temporal attention models for spiking transformers, such as STAtten and STSSA, extend self-attention into the temporal domain to better exploit the inherent temporal structure of spiking neural networks. STAtten uses non-overlapping blocks that contain groups of timesteps. These timesteps are compared within each block to provide segmented temporal attention. Alternatively, STSSA compresses the spatial dimension via max-pooling into representative temporal tokens, then applies a k-order temporal window to restrict each timestep to only access current and recent information. By introducing overlapping temporal blocks into STAtten, a new model (obSTAtten) is proposed that reduces the strict partitioning of non-overlapping block-wise attention while remaining within the STAtten framework. The attention model obSTAtten conditionally improves upon or rivals the performance of STAtten when tested with static input via the CIFAR10 dataset, and requires roughly the same training time in most cases. There is a bigger benefit to the use of obSTAtten in LIF models when compared with PLIF models.

1. Introduction

Artificial neural networks and by extension much of the field of deep-learning have been inspired by the human brain's ability to learn through exposure to information[1]. Learned models are capable of astounding feats once thought only possible by a human being, from outperforming humans in strategic games to outperforming clinical specialist in diagnostic tasks[1]. Despite this direct inspiration, classical artificial neural networks (ANNs) do not work the same way as the human brain, as they rely on held floating point values to model their neural activation, and they use non-biological learning rules[5]. This gap between biological neurons and artificial neurons due to structural and behavioral differences has been partially closed due to research into spiking neural networks[5].

1.1. Energy Usage Considerations

Spiking neural networks (SNNs) communicate via discrete, finite duration spikes[6][9] making them particularly suited at event based processing while allowing them to consume less energy if given specialized hardware (see Section 1.2.1) [10] as well as allowing them to avoid performing repeated MAC operations[9].

1.2. Neuromorphic Computing

Neuromorphic computing is broadly an attempt to replicate the computational behavior and outcomes of the human brain, with an aim to reduce the energy cost of deploying learning models[1][8]. These techniques are not without their flaws, SNNs typically suffer from limited representational capacity and lower classification accuracy when compared to conventional models[8]. Neuromorphic computing relies on specialized hardware and sensors as well as specialized algorithms, both of which are explored below.

1.2.1. Neuromorphic Hardware

Neuromorphic hardware is designed for the use case of deploying SNNs, and takes advantage of said SNN's high informational sparsity to reduce data movement on-and-off of the neuromorphic chip[1]. Using neuromorphic hardware SNNs often achieve significant reductions in energy usage when compared to the performance a similar task on typical hardware[1].

1.2.2. Neuromorphic Sensors

Neuromorphic sensors, like neuromorphic hardware, are inspired by the mode-of-operation of biological systems. These sensors produce output spikes when their inputs change as opposed to providing a continuous stream of information[1].

1.2.3. Neuromorphic Algorithms

Algorithms that learn to make sense of information encoded as spikes are called SNNs[1]. SNNs work with single bit binary activations over time instead of floating point activations like traditional ANNs use, allowing high spatial and

temporal sparsity (each neuron is usually not activated as activation is boolean)[1].

1.3. This Paper's Contribution

This paper contributes a modified version of the STAtten spatial-temporal attention model[6] for spiking transformers which attempts to increase the model's ability for temporal correlation by allowing overlapping temporal blocks. This model is referred to as overlapping-block STAtten (obSTAtten), and was experimentally compared with both STAtten and a 'default attention model' for static input over $T = 4$. In summary, our contributions are:

- This paper proposes obSTAtten, a modified spatial-temporal attention model for spiking transformers that supports overlapping temporal blocks.
- This paper evaluates obSTAtten against STAtten and a 'default attention model' over $T = 4$ static inputs.

For implementation details see Section 3 and for the outcome of our experiments see Section 5.

2. Related Works

While spiking neural networks have existed for some time, recent advances have significantly expanded their capabilities, particularly in the domain of visual attention. The models and mechanisms most relevant to this paper's contribution are summarized below.

2.1. Spiking Neural Networks

As opposed to traditional neural network implementations, spiking neural networks (SNNs) incorporate artificial neuron models which behave significantly more like biological neurons, allowing the resulting networks to achieve a higher degree of energy efficiency in use as well as more accurate interpretation of biology-inspired (neuromorphic) sensors[1]. Though also inspired by the brain, ANNs differ in computation mode, learning rules, and activation type to biological neurons.[5]

These models have demonstrated real-world success in medical applications such as anomaly monitoring and brain-machine interfacing[1]. These models have also shown wide usage in robotics both to enable more human-like behavior and to lower computational costs[1]. In similar fashion unmanned vehicles have also seen various implementations of SNNs due to their potentially low energy usage[1]. Despite the widespread utility of these models, a key challenge with implementing them in many cases is the inherent difficulty and computational expense in training them[5].

2.2. Leaky Integrate-and-Fire Model

One of the simplest and most foundation neuron models in computational biology and spiking neural network research is the leaky integrate-and-fire neuron model. The model can be described by its subthreshold dynamic function, shown in (1)[2].

$$\tau \frac{dV(t)}{dt} = -\left(V(t) - V_{rest}\right) + X(t) \tag{1}$$

In this model, $V(t)$ represents the membrane potential of the LIF neuron at time t, where $X(t)$ represents the corresponding input at time t. The constant τ is known as the membrane time constant, and represents the rate at which the current membrane potential decreases towards the resting membrane potential V_{rest}. When $V(t)$ exceeds a certain threshold V_{th} (a hyperparameter) the neuron outputs a spike-value of 1 for time t before instantaneously returning to $V(t) = V_{reset}$ where $V_{reset} < V_{th}$[2].

This model can be represented as an activation function like the one shown in (2) and (3)[6]. In this representation $u[t]^l$ is the membrane potential in the l-th layer at time t, and W^l is the weight of the l-th layer[6].

$$u[t+1]^l = \tau u[t]^l + W^l f(u[t]^{l-1}) \tag{2}$$

$$f(u[t]^l) = \begin{cases} 1, & \text{if } u[t]^l > V_{th} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

2.3. Parametric Leaky Integrate-and-Fire Model

The membrane time constant τ (often $\tau \in (0, 1]$[6]) is typically regarded as a global hyperparameter[2] similarly to the learning rate α in ANNs. In parametric leaky integrate-and-fire (plif) neurons τ is a trainable parameter that each neuron fine-tunes alongside their weights. A plif SNN is defined by the following properties:[2]

1. The time constant τ is automatically set during model training
2. The membrane time constant τ is shared within each layer of the model, which is biologically plausible (and is conveniently compatible with the activation function shown in (2) and (3))
3. The membrane time constant τ is not shared between layers, allowing diverse phase-frequency responsiveness

2.4. The Vision Transformer (ViT)

Classically, computer vision problems are often solved using convolutional neural networks (CNNs) which have a limited kernel size and thus struggle to correlate input data that is non-contiguous. Transformers are an alternative solution to this architectural limitation, where the transformer contains an attention model which can be used to correlate non-contiguous input data. This is done for visual inputs by creating P patches by splitting the image, and from them N tokens. Multiple attention heads that train to correlate different projections (Query, Key and Value) are used to determine correlations between patches using the transformer's attention model.

2.5. Standard Self Attention

An image $x \in \mathbb{R}^{W \times H \times C}$ of width W, height H and C channels is input into the model and split into P patches. This generates N tokens (4), allowing each patch to be flattened into a trainable embedding vector (5)[8]. From this $x_p^i \in \mathbb{R}^{(P^2C)}$ is the i-th patch, $E \in \mathbb{R}^{(P^2C) \times D}$ is a learnable projection matrix and the matrix E_{pos} denotes positional embeddings[8].

$$N = \frac{WH}{P^2} \tag{4}$$

$$z_0 = [x_p^1\mathbf{E}; x_p^2\mathbf{E}; ...x_p^N\mathbf{E};] + E_{pos} \tag{5}$$

This input spike sequence can be used to generate three projections; $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ which represent the Query, Key and Value[8] projections of the input. For a floating-point input tensor X_f with N tokens and D features (6), the floating point attention projections $\mathbf{Q_f}$, $\mathbf{K_f}$ and $\mathbf{V_f}$ can be found using the trained weights $\mathbf{W_Q}$, $\mathbf{W_K}$ and $\mathbf{W_V}$ (7) where $\mathbf{W_Q}, \mathbf{W_K}, \mathbf{W_V} \in \mathbb{R}^{D \times D}$.

$$X_f \in \mathbb{R}^{N \times D} \tag{6}$$

$$Q_f = W_Q X_f, \quad K_f = W_K X_f, \quad V_f = W_V X_f \tag{7}$$

Using the floating point projections $\mathbf{Q_f}$, $\mathbf{K_f}$ and $\mathbf{V_f}$, standard self attention can be modeled as shown in (8)[8]. Notice that this attention model does not have a dimension of time and requires floating point input. Without modification this cannot be used for spiking neural networks.

$$\text{Atn} = \text{Softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{D}}\right)V, \quad Q_f, K_f, V_f \in \mathbb{R}^{N \times D} \tag{8}$$

2.6. Spiking Self Attention

Spiking neural networks cannot directly process floating point input data such as X_f, thus for a binary tensor $X \in \mathbb{R}^{\mathbb{T} \times \mathbb{N} \times \mathbb{D}}$ for timestep T, spiking self attention (SSA) can be found via (10) for $\mathbf{Q}, \mathbf{K}$ and $\mathbf{V}$ (9). Note that α is a scaling factor and is not the learning rate (as it would be for an ANN).

$$\mathbf{Q}, \mathbf{K}, \mathbf{V} = \text{LIF}(\mathbf{W_Q X}), \text{LIF}(\mathbf{W_K X}), \text{LIF}(\mathbf{W_V X}) \tag{9}$$

$$\text{SSA}(\mathbf{QKV}) = \text{LIF}(\mathbf{QK}^T\mathbf{V} \cdot \alpha) \tag{10}$$

2.7. Spatial-Temporal Attention (STAtten)

For a binary input tensor $X \in \mathbb{R}^{\mathbb{T} \times \mathbb{N} \times \mathbb{D}}$ over T timesteps, the spiking neural network has atleast T timesteps to settle on its final output. Since a spiking neural network necessarily operates in time (unlike an ANN) it is potentially useful to expand its self attention model to include temporal attention. If the timesteps in T are partitioned into non-overlapping blocks B, a block-wise spatial-temporal attention model (STAtten) can be formed. The model for STAtten can be seen in (11), where $[b]$ is defined in (12)[6].

$$\text{STAtten}(X[b]) = \text{LIF}(\mathbf{Q}[b]\mathbf{K}^T[b]\mathbf{V}[b] \cdot \alpha) \tag{11}$$

$$[b] = [iB : (i+1)B, :, d], \quad i \in \{0, 1, ..., T/B - 1\} \tag{12}$$

The notation $[iB : (i+1)B, :, d]$ represents the aggregated features from timesteps iB to $(i+1)B$[6]. As an example, for $T = 4$ and $B = 2$, this will create blocks $[0, 1]$ and $[2, 3]$ where the features contained within paired timesteps are compared while other features are not, making computation manageable[6]

2.8. Spatial Temporal Spiking Self-Attention (STSSA)

Alternatively to the block-wise approach taken by STAtten, pooling can be used to collapse the temporal dimension before being fed into the attention model. The three differences between SSA and STSSA are representative spiking temporal tokens, temporal window masking and spiking spatial-temporal feature fusion[11].

2.8.1. Representative Spiking Temporal Tokens (RSTT)

For the binary input tensor $X \in \mathbb{R}^{\mathbb{T} \times \mathbb{N} \times \mathbb{D}}$, the simplest processing approach would be to treat the entire object as a two-dimensional matrix where T and N are dimensionally combined[11], however passing this into SSA would yield a complexity of T^2N^2D which is computationally intractable. Since SNNs are temporally sparse (they produce discrete spikes at only some time-steps) the temporal dimension can be collapsed using max pooling (MP) as shown in (13)[11].

$$R = \text{LIF}(\text{MP}(W_Q X)) \tag{13}$$

This results in $R \in \mathbb{R}^{T \times n \times D}$ where $n << N$.[11] This results in n representative tokens, which represent a compression of the initial temporal information.

2.8.2. Temporal Window Masking

If all R over T timesteps were directly aggregated the resulting system would suffer delays and excess computational requirements[11]. The chosen solution is to mask R

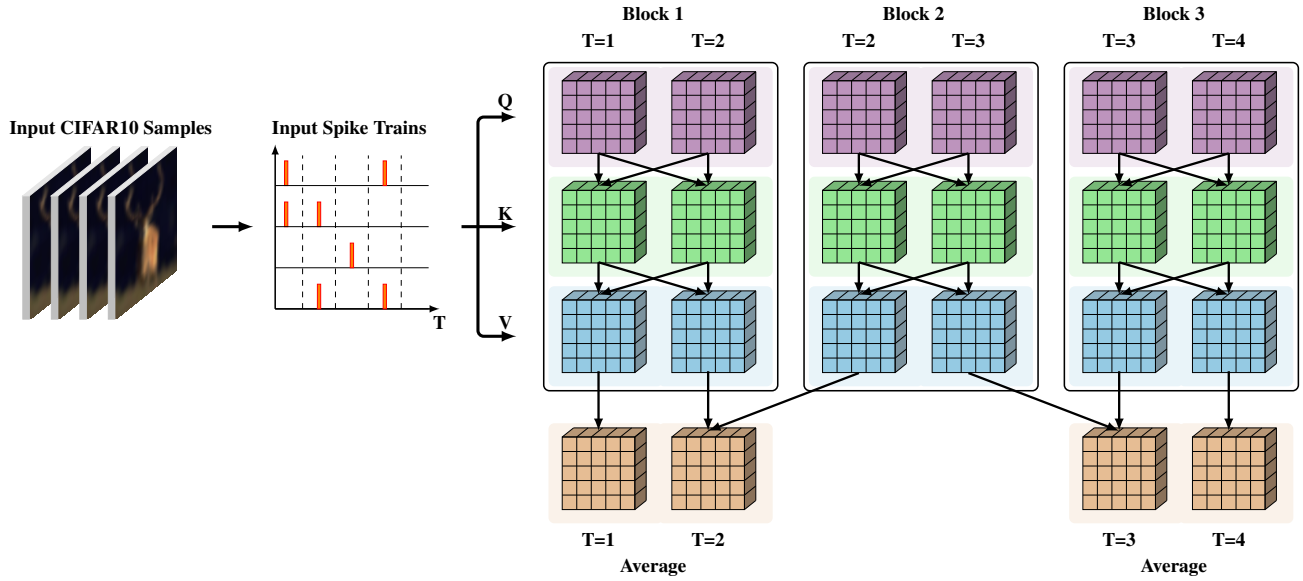

Figure 1. obSTAtten Attention Model showing determined block count and temporal output averaging

for k timesteps such that $Q[t]$, $K[t]$ and $V[t]$ can only access information from the current timestep and the previous $k-1$ timesteps[11].

2.8.3. Spiking Spatial-Temporal Feature Fusion

The k-order mask developed above is applied to RSTT resulting in $\mathbf{R_M}$. $\mathbf{R_M}$ is inserted between Q and K[11] resulting in the model STSSA, shown in (14).

$$\text{STSSA}(\mathbf{Q}, \mathbf{R_M}, \mathbf{K}, \mathbf{V}) = \text{LIF}((\mathbf{Q}\mathbf{R}_M^T)(\mathbf{R}_M\mathbf{K}^T)V * S) \quad (14)$$

2.9. SpikingJelly

SpikingJelly is an open source machine learning infrastructure platform dedicated to spiking neural networks[3]. This unified framework for developing SNNs is utilized by the code-base of STAtten[6] and is thus utilized by the code-base of obSTAtten.

3. Proposed Approach

This section will outline the modifications made to the STAtten attention model[6] and provide explanation for the workings of the obSTAtten attention model. The core concepts covered are how obSTAtten determines the number of temporal blocks to use, how overlapping temporal blocks are constructed and the averaging of the resulting temporal outputs. As discussed in Section 3.3 both STAtten and obSTAtten use the came method for computing attention from said temporal outputs.

3.1. Determining Block Count

In unmodified STAtten the number of temporal blocks N_B can be easily determined by the number of timesteps T and the block size S_B as shown in (15), where $//$ denotes the floor division operator. For $T = 4$ and $S_B = 2$, there should be $N_B = 2$ temporal blocks. This is because in STAtten blocks are non-overlapping temporal partitions, so each timestep t can only appear in one block.

$$N_B = T//S_B \quad (15)$$

In obSTAtten the number of temporal blocks N_B is slightly more complex to calculate. This complexity results from the allowance of overlapping temporal blocks which no longer create non-overlapping temporal partitions. For simplicity the model has been configured for an overlap of $T_{ov} = 1$ and a stride $s = 1$, resulting in the relationship shown in (16). As before, N_B is the number of blocks, T is the number of time-samples t and S_B is the block size.

$$N_B = T - S_B + 1 \quad (16)$$

For an example of $T = 4$ and $S_B = 2$, there should be $N_B = 3$ blocks, as shown in Figure 1. In Figure 1, the generated $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ projections at each duplicate timestep are shared between both blocks.

3.2. Overlapping Block Construction

For T time samples and N_B blocks, overlapping block $\mathcal{B}_i$ is formed by (17) for index i. Note that if the index i had a stride $s = S_B$ this would output the same set of blocks as STAtten.

$$\mathcal{B}_i = [i : i = N_B, :, d], \quad i \in 0, 1, ..., T - S_B \tag{17}$$

3.3. Attention Computation

After the generation of N_B overlapping blocks $\mathcal{B}_i$, obSTAtten computes its attention identically to how STAtten does. Since the output dimensions of the overlapping block construction outlined above are the same as the non-overlapping blocks produced by STAtten ($S_B \times N \times D$), the rest of the model was left unchanged. As a result the obSTAtten attention model can be seen in (18).

$$\text{obSTAtten}(X_{\mathcal{B}_i}) = \text{LIF}(\mathbf{Q}_{\mathcal{B}_i}\mathbf{K}_{\mathcal{B}_i}^T\mathbf{V}_{\mathcal{B}_i} \cdot \alpha) \tag{18}$$

3.4. Temporal Output Averaging

Let the final model output at timestep t be $\hat{Y}_t$ where $i : t \in \mathcal{B}_i$ is the set of overlapping blocks containing timestep t. Since multiple blocks are contributing to $\hat{Y}_t$ their contributions must be averaged. This is done via (19).

$$\hat{Y}_t = \frac{1}{|i : t \in \mathcal{B}_i|} \sum_{i:t\in\mathcal{B}_i} \text{obSTAtten}(X_{\mathcal{B}_i})_t \tag{19}$$

Note that timesteps $t = 0$ and $t = T - 1$ are not averaged because they each only appear in one block. This behavior is shown in Figure 1.

4. Experiments

The experiments performed in this paper focused on using CIFAR10 (static images) as input to train various permutations of STAtten and obSTAtten such that their accuracy scores can be evaluated. The dataset used (CIFAR10) and a considered dataset (CIFAR10-DVS) are expanded on below.

4.1. Datasets

There are two datasets which were explored when running experiments for this paper, being CIFAR10 and CIFAR10-DVS. Details about each dataset and the key differences between them are expanded on below.

4.1.1. CIFAR10

CIFAR10 is a dataset that contains 60,000 32×32 color images in ten classes (6,000 images per class). The test batch contains 1000 randomly selected images from each class. The training batches contain a non-uniform distribution of the remaining 5,000 images per class. The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck[4]. This is the single dataset used in experimentation for this paper.

4.1.2. CIFAR10-DVS

CIFAR10-DVS is a dataset much like CIFAR10, however the dataset contains DVS (dynamic vision sensor, a type of neuromorphic sensor) recorded samples of CIFAR10 in 128×128 resolution over many timesteps. This allows the input of the target SSN to change each timestep during training[7]. This should provide more realistic training for use with neuromorphic hardware and sensors (eg in robotics)[1]. Though this would have made an excellent benchmark for obSTAtten, a study of CIFAR10-DVS could not be run due to hardware limitations (see Section 4.2).

4.2. Hardware Limitations

The experiments run to collect data for obSTAtten were limited by severe hardware limitations. The source code for STAtten and SpikingJelly requires a CUDA[3] capable GPU for training. The only machine at the disposal of this project was a moderately old desktop gaming PC that contains an Nvidia GTX 1070TI which limited training to a batch-size of 4 and prevented any training parallelism from being implemented. This likely significantly slowed down training as well reducing the number of experiments that could be run during the research time.

Another limitation is storage space, the machine that was used to train obSTAtten has very limited (and slow) storage space, preventing CIFAR10-DVS from being trained on. This limited the experiment to using CIFAR10, which did not allow for a comparison between static and DVS input for obSTAtten.

4.3. Training Conditions

For four conditions, both STAtten, obSTAtten, and a non-temporal standard spiking transformer model (SDT) were trained allowing their performance to be compared. These conditions had the model dimension d tested for $d = 256$ and $d = 512$, as well as training using LIF neurons vs PLIF neurons. These four conditions were trained for all three attention models, resulting in 12 complete training runs. Each run trained for 210 epochs, as this is the default configuration when training STAtten on CIFAR10 meaning the training conditions for both models were the same. The model architecture used was the provided Spiking Transformer mode based on SpikingJelly[6][3].

5. Results

After completing 12 210-epoch training runs on three attention models (SDT, STAtten and obSTAtten) for four parameters some interesting patterns have been observed.

5.1. Tabulated Results

When comparing best Top-1 accuracy the default attention model (SDT) did surprisingly well when the model dimension was set to $d = 256$. This was an unexpected result,

however in conjunction with another piece of data it tells an interesting story. For the $d = 256$, LIF training run notice that obSTAtten and STAtten perform identically, and they peak at the same epoch. This was a suspicious result, as such the obSTAtten condition was trained twice to be sure, and the result was the same both times. Both of these results hint towards the idea that when $d = 256$, the model is not large enough to benefit from richer temporal context as its main accuracy bottleneck is its size. In these cases there is no clear reason to use spatial-temporal attention of either kind for static input. It should be noted that in the $d = 256$ PLIF case obSTATTen did outperform STAtten.

Table 1. Dim-256 LIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	93.08%	**93.13%**	**93.13%**
Best Top-1	**93.33%**	93.30%	93.30%
Best Epoch	205	**199**	**199**
Training Time	17.10 hrs	**16.99 hrs**	17.26 hrs

Table 2. Dim-256 PLIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	**93.44%**	93.32%	93.41%
Best Top-1	**93.61%**	93.32%	93.48%
Best Epoch	**204**	209	209
Training Time	**18.84 hrs**	18.96 hrs	19.33 hrs

When the model dimension is increased to $d = 512$ more interesting behavior can be observed. When trained using LIF neurons obSTAtten clearly outperforms STAtten, peaks at an earlier epoch and trains slightly faster. When trained using PLIF neurons however, STAtten slightly outperforms obSTAtten and trains slightly faster, but obSTAtten still peaks at an earlier epoch.

Table 3. Dim-512 LIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	**94.42%**	**94.42%**	94.28%
Best Top-1	94.50%	94.50%	**94.76%**
Best Epoch	208	208	**206**
Training Time	28.74 hrs	28.47	**28.30 hrs**

Table 4. Dim-512 PLIF Results

Attention	SDT	STAtten	obSTATTen
Last Top-1	94.29%	**94.62%**	93.26%
Best Top-1	94.57%	**94.63%**	93.34%
Best Epoch	200	208	**194**
Training Time	**30.34 hrs**	30.47 hrs	30.67 hrs

5.2. 512-dim, LIF Training Times per Epoch

When STAtten and obSTAtten were trained on CIFAR10 for the conditions 512-dim and LIF neurons, obSTAtten comparatively did the best of the four cases. During this experiment the training time per epoch for all three attention models is plotted in Figure 2. It can be seen around epoch 140 obSTAtten becomes slightly slower-to-train than STAtten, however it quickly recovers and never trains as slowly as SDT did. After its recovery it appears to have a similar training rate to STAtten, showing it only has significantly faster training times in the first 100 or so epochs.

5.3. 256-dim, LIF STAtten obSTAtten Convergence

As noted in Section 5.1, when STAtten and obSTAtten were trained on CIFAR10 for the conditions 256-dim and LIF neurons, they obtained the same best Top-1 score at the same epoch, however from Figure 3 it can be seen that the two models arrive at this converged point via the same exact steps. In this test case, the two models are actually equivalent, and obSTAtten behaves identically to STAtten. As noted before this test was repeated because this was assumed to be an error but it was replicated twice.

One possible explanation for this behavior is that the 256-dim LIF model is significantly smaller and has less trainable parameters than any of the others. This model might simply be too simple to benefit significantly from spatial-temporal attention, and the repeat behavior being seen is merely the model bottle-necking at its maximum performance for both cases.

Another possible explanation for this behavior is that the 256-dim LIF model simply cannot represent enough distinct features from its input for the improved temporal context provided by obSTAtten to have any effect.

5.4. LIF vs PLIF Effect on obSTAtten Performance

An interesting observation is that obSTAtten performs better with LIF neurons than it does with PLIF neurons. PLIF neurons have another degree of learning via their trainable parameter τ, which is a time-constant and thus effects the temporal behavior of the neuron[2]. Since PLIF neurons can already encode temporal information this way, the new information provided by obSTAtten may have been redundant which could have confused the network or introduced extra noise.

Consider how STAtten did during the two PLIF experiments, when dimension $d = 256$, the non-temporal attention model performed the best in all metrics. When dimension d was increased to 512, STAtten only did marginally better than the non-temporal attention model and took 8 more epochs to get there. STAtten also performs worse when trained using PLIF neurons when compared to the non-temporal attention model.

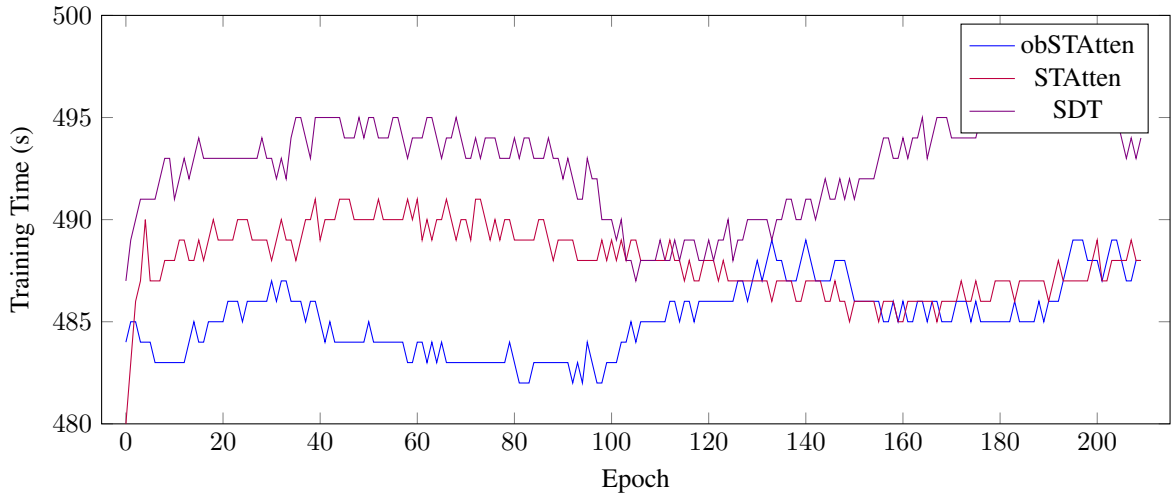

Figure 2. 512-dim, LIF Model train time on CIFAR-10

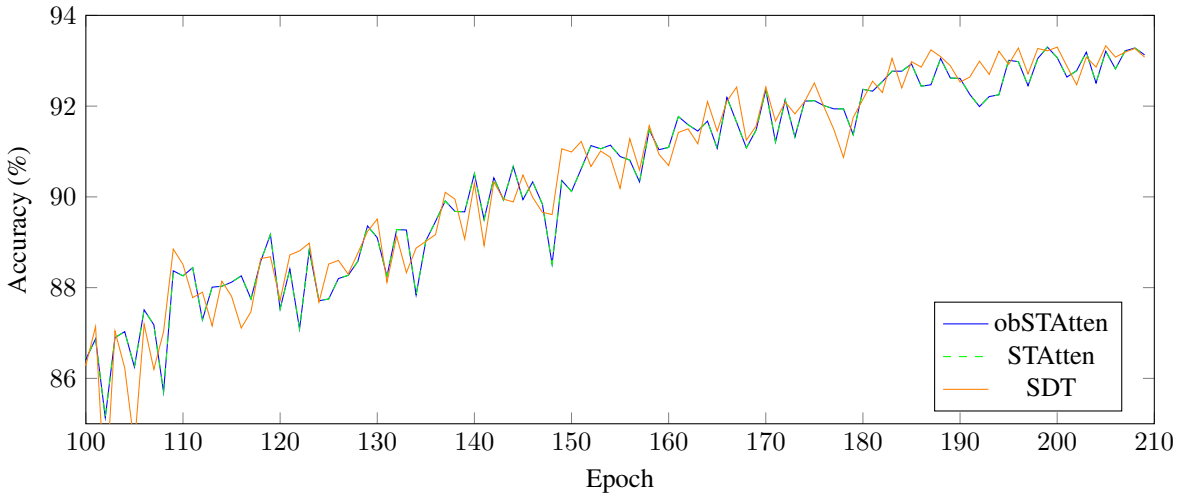

Figure 3. 256-dim, LIF Model Test accuracy on CIFAR-10

5.5. STAtten vs obSTAtten for Best-Epoch

It can be seen in both experiments with dimension $d = 512$ that obSTAtten reaches its best epoch sooner than STAtten does. This implies that the extra temporal context provided by obSTAtten's overlapping temporal blocks allows the model to learn faster initially when the scheduled learning rate is higher[6]. During this early period the model is making coarser adjustments to its learned parameters, and the small amount of extra temporal context obSTAtten provides results in a stronger gradient signal. When the learning rate decreases as the model matures this effect also shrinks. This is a metric other than Top-1 in which this model has conditional improvements over STAtten, though similar to improvements in Top-1 accuracy, this only occurs when the model is sufficiently complex.

6. Conclusion

In this paper, a spiking transformer attention model called obSTAtten was developed from the model STAtten[6], which aimed to allow for overlapping temporal blocks while still utilizing the mathematical framework of STAtten. This required making modifications to STAtten, resulted in key findings, and has produced two interesting avenues for future work. This paper has some limitations which are also mentioned below.

6.1. Summary of Contribution

Producing an attention model obSTAtten from the STAtten code-base required implementing three major features, as detailed below.

6.1.1. Determining Overlapping-Block Count

A new and slightly more complex approach to determining the number of temporal blocks (N_B) was developed to allow overlapping blocks and shared timesteps. This was implemented for a stride $s = 1$ and an overlap $T_{ov} = 1$ for simplicity and reliability in testing.

6.1.2. Overlapping Block Construction

Next N_B overlapping blocks $\mathcal{B}_i$ for index i are generated of the form $[i : i = N_B, :, d]$. These blocks are the same shape as the blocks generated by STAtten allowing the rest of existing framework to utilize these blocks with no extra modification.

6.1.3. Temporal Output Averaging

The model output $\hat{Y}_t$ for timestep t requires the contributions of each timestep t to be averaged before they are utilized. This step does not apply to the first or last timestep $t = 0$ or $t = T - 1$ as these timesteps only exist on one block.

6.2. Key Findings

There were several notable findings that resulted from the experiment detailed in this paper, the details of each one will be expanded on below.

6.2.1. Model Complexity Effect on obSTAtten Performance

A model's complexity can be derived from the dimension of its trainable parameters (256 or 512) and by the presence of parameterization of the membrane time potential τ (LIF or PLIF). Models that lack sufficient complexity do not seem to benefit from obSTAtten and they do not significantly benefit from STAtten either when using static image inputs (CIFAR10).

6.2.2. LIF vs PLIF Effect on obSTAtten Performance

PLIF neurons have learnable membrane time constant τ allowing them to learn a temporal parameter. This feature already allows PLIF neurons to correlate temporal information, as such the extra temporal information provided by obSTAtten is either redundant, confusing or noise inducing which lowers the test accuracy of the model.

6.2.3. Training Times per Epoch

When STAtten and obSTAtten were trained on CIFAR10 for the conditions 512-dim and LIF neurons, obSTAtten outperforms STAtten. During this experiment it can be seen around epoch 140 that obSTAtten becomes slightly slower-to-train than STAtten, but it catches back up. It is only significantly faster at training before this event.

6.2.4. 256-dim LIF Convergence

When the least complex model was trained, the STAtten and obSTAtten model behavior is identical, meaning for models of low enough complexity STAtten and obSTAtten are computationally equivalent (but not equal in training speed). This was run twice to ensure it was not an error.

6.2.5. STAtten vs obSTAtten for Best-Epoch

STAtten seems to take more epochs to reach its best epoch than obSTAtten does in most cases, suggesting the improved temporal context does improve early learning rates when adjustments to the network are coarser. This effect settles off as the model matures towards epoch 210.

6.3. Limitations

Due to hardware constraints and time constraints, only CIFAR10 was used in training, as such none of the experiments were run using DVS or other dynamic inputs. It is possible that such dynamic input would have made the two models behave in ways not replicatable using static image input.

6.4. Future Work

Future work can take two possible avenues, the first would be a deeper dive into the current state of obSTAtten by allowing more hyper-parameter tuning than the model currently does. The second option would have obSTAtten train on different datasets such as CIFAR100[4] and CIFAR10-DVS.

6.4.1. Stride and Overlap Adjustment

In the current implementation of obSTAtten, stride and overlap for temporal blocks are both hard-coded values, adjusting them may yield interesting results and should be explored in future work.

6.4.2. CIFAR100 and CIFAR10-DVS

Currently hardware and storage space limitations prevent the usage of larger datasets like CIFAR10-DVS which may yield interesting results that static datasets such as CIFAR10 or CIFAR100 do not. This may be useful to study for obSTAtten because SNNs are temporally dynamic unlike ANNs.

References

[1] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 111(9):1016–1054, 2023. 1, 2, 5

[2] Wei Fang, Zhaofei Yu, Yanqi Chen, Timothée Masquelier, Tiejun Huang, and Yonghong Tian. Incorporating learnable membrane time constant to enhance learning of spiking neural networks. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 2641–2651, 2021. 2, 6

[3] Wei Fang, Yanqi Chen, Jianhao Ding, Zhaofei Yu, Timothée Masquelier, Ding Chen, Liwei Huang, Huihui Zhou, Guoqi

Li, and Yonghong Tian. Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence. *Science Advances*, 9(40):eadi1480, 2023. 4, 5

[4] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, Department of Computer Science, University of Toronto, 2009. 5, 8

[5] Himanshu Kumar and Aniket Konkar. Simple transformer with single leaky neuron for event vision. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, pages 865–871, 2025. 1, 2

[6] Donghyun Lee, Yuhang Li, Youngeun Kim, Shiting Xiao, and Priyadarshini Panda. Spiking transformer with spatial-temporal attention. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13948–13958, 2025. 1, 2, 3, 4, 5, 7

[7] Hongmin Li, Hanchao Liu, Xiangyang Ji, Guoqi Li, and Luping Shi. CIFAR10-DVS: An event-stream dataset for object classification. *Frontiers in Neuroscience*, 11:309, 2017. 5

[8] Yiqin Xiang, Tee Hui Teo, Junwen Zhang, and Hanzhi Ma. Vision spiking transformer for image classification. In *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC)*, pages 327–330, 2025. 1, 3

[9] Yichen Xiao, Shuai Wang, Dehao Zhang, Wenjie Wei, Yimeng Shan, Xiaoli Liu, Yulin Jiang, and Malu Zhang. Rethinking spiking self-attention mechanism: Implementing α-xnor similarity calculation in spiking transformers. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5444–5454, 2025. 1

[10] Peixiang Zheng and Zhao Shen. Optimizing spiking neural networks with transformer-based spatial attention. In *2024 4th International Symposium on Computer Technology and Information Science (ISCTIS)*, pages 82–85, 2024. 1

[11] Zhaokun Zhou, Jun Niu, Yang Zhang, Li Yuan, and Yuesheng Zhu. Spiking transformer with spatial-temporal spiking self-attention. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5, 2025. 3, 4

Modification to STAtten for Overlapping Temporal Blocks

Supplementary Material

GitHub Repository:https://github.com/rancorjoy/obSTAtten

7. Code Modifications

The following section details the modifications that were made to the original code-base of STAtten to add the attention model obSTAtten. All changes can be seen in the GitHub Repository linked above. Since it is hard to fit longer lines of code into CVPR format, tab indented lines of code should be assumed to belong to their parent (which won't be tab indented), to distinguish actual indentation from linebreaks, there will be no empty lines within a line of code. Naturally most of the comments do not fit inside to confines of the code blocks, to see the full and commented code please view the GitHub Repository linked above.

7.1. Modifications to train.py

Line 261 of train.py was replaced with the following code to ensure the option 'obSTAtten' for the parameter 'attention_mode' would be valid.

```
parser.add_argument("--attention_mode",
        type=str, default="STAtten",
        help='choose one of methods:
           STAtten, obSTAtten, SDT')
```

A few bugs were fixed as well, however these seem to have been rendered non-functional at some point in the past and they do not affect the functioning of obSTAtten. These lines of code are marked by comments in the Github Repository.

7.2. Modifications to ms_conv.py

First the line of code shown below asserts the available attention modes, another mode needs to be added to this list or else the code will return without running for any other option entered.

```
assert self.attention_mode not in
        ["STAtten, SDT"]
```

Into this list the string "obSTAtten" is added so that it becomes a valid attention argument. Now obSTAtten can be implemented.

```
assert self.attention_mode not in
        ["STAtten, obSTAtten, SDT"]
```

Next the conditional statement that forms the STAtten attention model selection is copied, it starts with the conditional statement "if self.attention_mode == "STAtten": ". This entire block is copied and the top comment and conditional string are swapped. Now if nothing else changes obSTAtten will run and perform exactly equivalent to STAtten.

```
## obSTAtten Attention Mode --
        "overlapping block STAtten"

        if self.attention_mode ==
                        "obSTAtten":

            if self.dvs:
                scaling_factor = ...
                1 / (H * H * ...
                self.chunk_size)

            else:
                scaling_factor = 1 / H
```

Finally edits can be made to the code to allow for overlapping blocks. First the number of code blocks needs to be corrected, there will be a line of code that looks like the one shown below:

```
# Vectorized Attention
num_chunks = T // self.chunk_size
```

This block should be replaced with the code shown below to create the correct number of blocks.

```
# This model produces number blocks ...
num_chunks = T - self.chunk_size + 1
```

Next the attention model combines all data in contiguous chunks, as shown below. This cannot be used in obSTAtten because overlapping blocks are not contiguous. To solve this the function view() is replaced with the function unfold().

```
# Reshape q, k, v to process all ...
q_chunks = q.view(num_chunks,
self.chunk_size, B, self.num_heads, N,
head_dim).permute(0, 2, 3, 1, 4, 5)

k_chunks = k.view(num_chunks,
self.chunk_size, B, self.num_heads, N,
head_dim).permute(0, 2, 3, 1, 4, 5)

v_chunks = v.view(num_chunks,
self.chunk_size, B, self.num_heads, N,
head_dim).permute(0, 2, 3, 1, 4, 5)
```

The function unfold() produces an object with the same dimensions however they are ordered incorrectly, to reorder them (such that they match the output of view()) the code shown below is used.

```
# Replace .view() with .unfold() ...
# .unfold() needs dimension, size ...
q_chunks = q.unfold
        (0, self.chunk_size, 1)

k_chunks = k.unfold
        (0, self.chunk_size, 1)

v_chunks = v.unfold
        (0, self.chunk_size, 1)

# shape: (num_chunks, B, num_heads, ...

# STAtten permutes (num_chunks, ...
# .unfold() puts chunk_size at ...
q_chunks = q_chunks.permute
        (0, 1, 2, 5, 3, 4)

k_chunks = k_chunks.permute
        (0, 1, 2, 5, 3, 4)

v_chunks = v_chunks.permute
        (0, 1, 2, 5, 3, 4)
```

The last modification to the attention algorithm is simply an insertion. There is a section near the bottom of STAtten/obSTAtten that contains the following code, the comment "EMPTY SPACE" is not present in the code but marks the space where the insertion needs to be made.

```
# Reshape back to separate temporal ...
out = out.reshape(num_chunks, B,
        self.num_heads,self.chunk_size,
        N, head_dim).permute
        (0, 3, 1, 2, 4, 5)
# Flatten chunks back to T: (T, ...
output = out.reshape
        (T, B, self.num_heads, N,
        head_dim)

# EMPTY SPACE

x = output.transpose(4,3).
        reshape(T, B, C, N).

        contiguous() # (T, B, head, ...
x = self.attn_lif(x).
        reshape(T, B, C, H, W)

if self.dvs:
        x = x.mul(x_pool)
           x = x + x_pool
```

The insertion is a block of code that will average all the context gathered for timestep across all instances of that timestep, the code is shown below. With the addition of this code obSTAtten should be functional, got to the GitHub Repository readme file to get instructions for running the code, as well as viewing the output log files visually using the included Jupyter Notebook.

```
# Unlike in STAtten where each time ...
# The output from each time-step  ...
output_acc = torch.zeros
        (T, B, self.num_heads,
        N, head_dim, device=x.device,
        dtype=out.dtype)

count = torch.zeros(T, device=x.device,
        dtype=out.dtype)

for i in range(num_chunks):
        # chunk i covers timesteps ...
        output_acc
                [i:i+self.chunk_size]
                += out[i].permute
                (3, 0, 1, 2, 4)

        count[i : i + self.chunk_size]
                += 1

        # Divide each timestep by ...
        # count shape: (T,) -> ...
        output = output_acc /
                count.reshape
                (T, 1, 1, 1, 1)
```

8. Unused Figures and Observations

From the gathered data several figures were generated which could not be used in the main paper. This is due to the stringent 8-page limit of the CVPR format paired with the fact that these figures are not all that interesting on their own. They do show that the behavior of each version of STAtten and obSTAtten is different, which confirms the code is running correctly in both cases.

8.1. obSTAtten Training Time per Epoch

Notice from the training time vs epoch plots (the first three) how obSTAtten is typically the slowest to train. This makes sense because its code is technically more complex than STAtten's code given that there is an insertion and both replacements are longer sections of code than the originals were. obSTAtten is only marginally slower to train whith dim-256, but when dim-512 the model is far slower (but only in the later epochs).

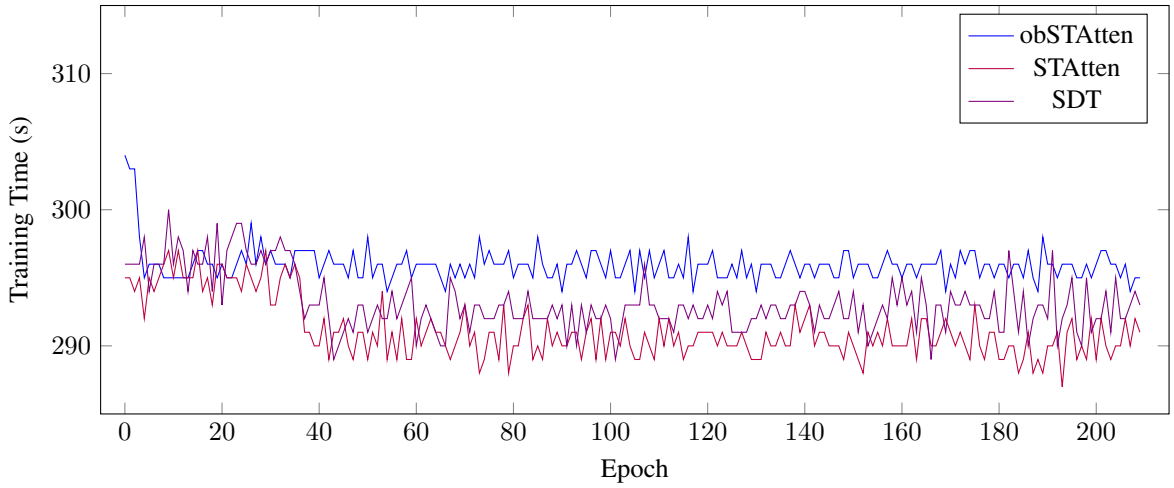

Figure 4. 256-dim, LIF Model train time on CIFAR-10

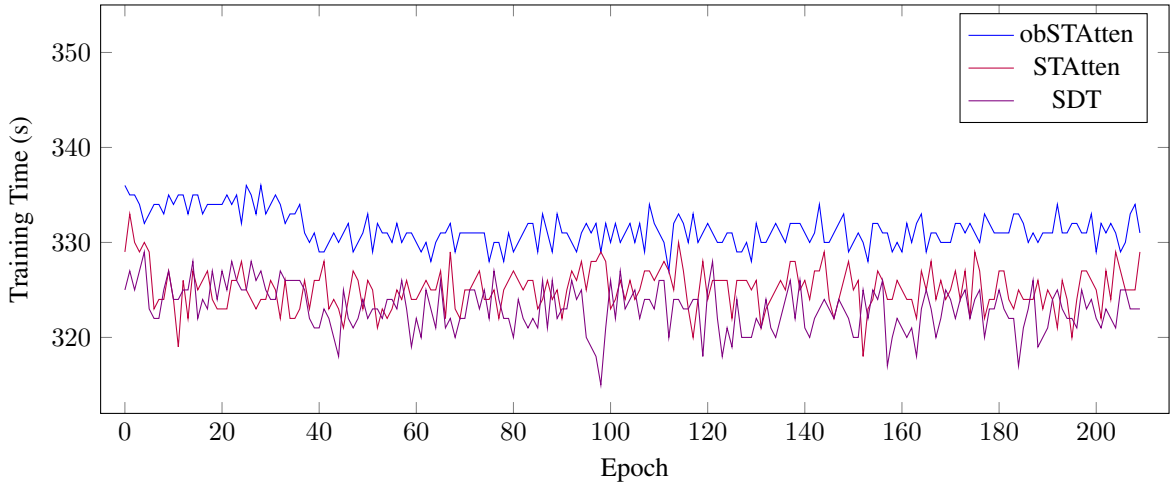

Figure 5. 256-dim, PLIF Model train time on CIFAR-10

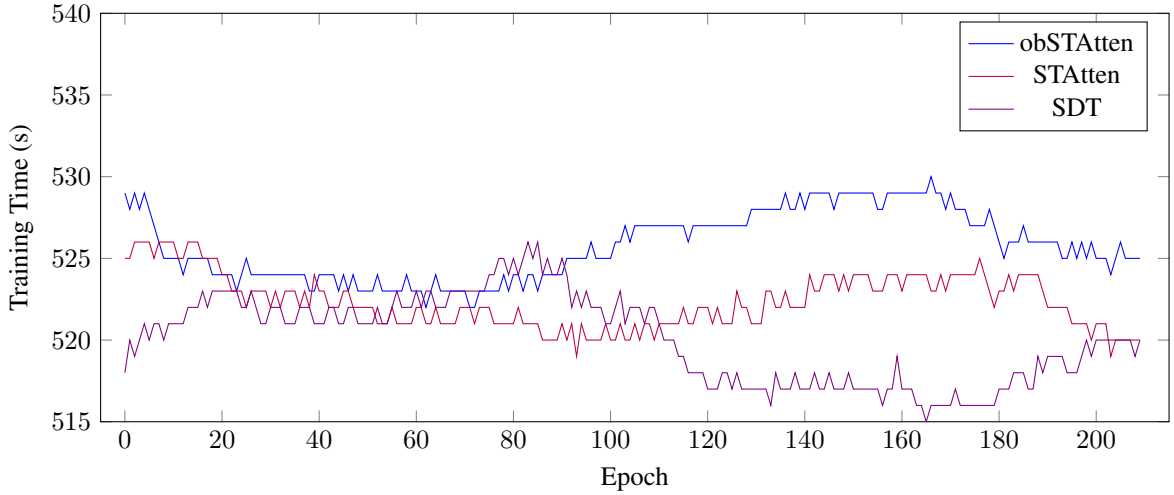

Figure 6. 512-dim, PLIF Model train time on CIFAR-10

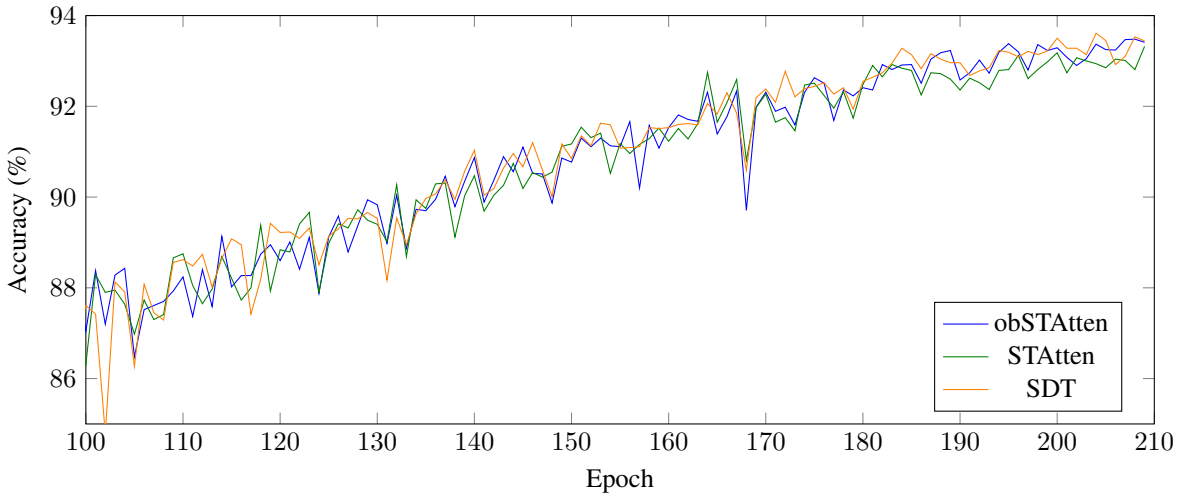

Figure 7. 256-dim, PLIF Model Test accuracy on CIFAR-10

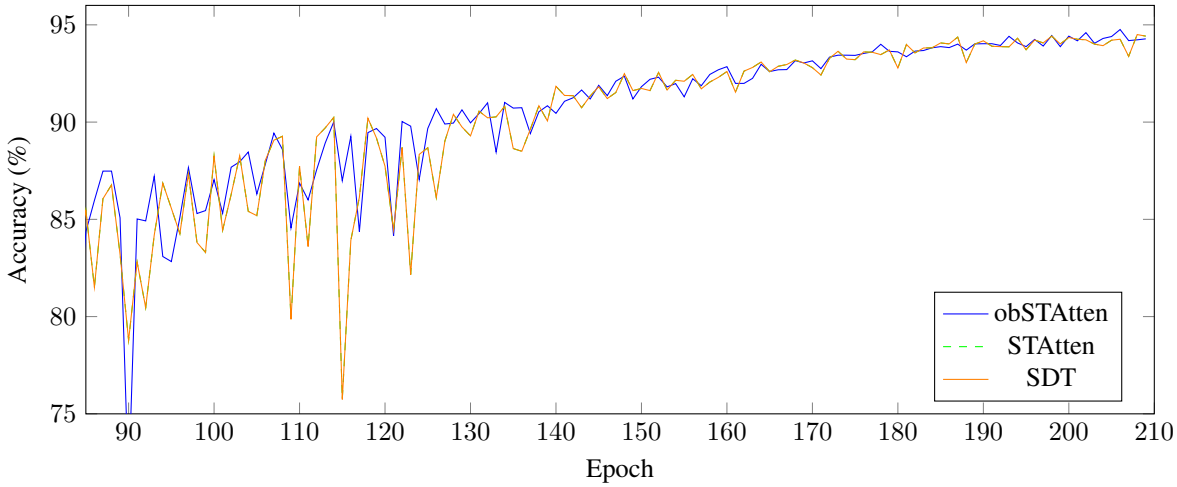

Figure 8. 512-dim, LIF Model Test accuracy on CIFAR-10

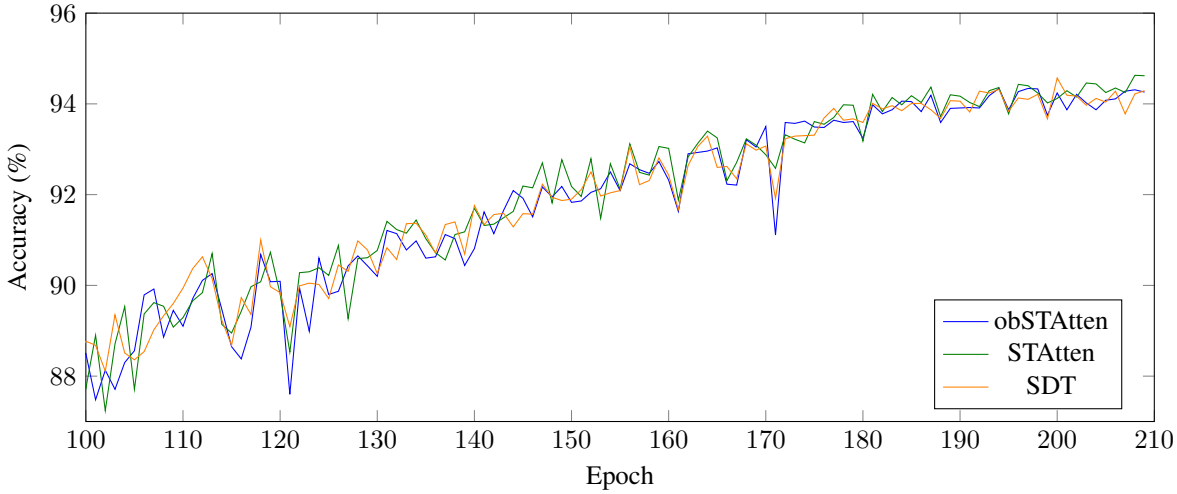

Figure 9. 512-dim, PLIF Model Test accuracy on CIFAR-10